D3 hardware test guide

ESP32-S3 / one HC-SR04+ / LD2450
Mac walkthrough - d3-board - 9 November 2026

READ BEFORE STARTING

Prepared for `d3-board` on 9 November 2026. These are procedures to run, not recorded passing results. The host revision does not by itself prove which firmware is on the board; retain the build and flash output too.

Use the documented **3.3 V-compatible HC-SR04+**. Its direct Echo connection requires no resistor. A standard 5 V Echo module needs voltage protection. Follow the printed pin labels, not wire colours. Unplug USB before changing connections. This guide starts with a bare ESP32-S3, not the team's shared sensor/LCD breadboard.

The tests clear volatile sensor/rule configuration and may reset occupancy. Keep application flash logging disabled. No external-flash programming or erase is needed.

1. Prepare the Mac and record the revision

Commands labelled as console commands go to the ESP32 monitor; shell commands go to Mac Terminal. Long commands wrap for printing: enter each BEGINCFG through ENDCFG as one continuous line, pressing Enter only at the end.

Open Terminal and run:

MAC TERMINAL

```
cd /Users/tanyilin/Downloads/EMBSYS-P2-AAI15
git status --short --branch
git branch --show-current
```

The branch should be `d3-board`. Do not switch branches with unrelated uncommitted work. The existing pinned ESP-IDF installation and `.venv` are assumed; if missing, follow README Part A first.

Create a fresh evidence folder in this Terminal:

MAC TERMINAL

```
D3_RUN="parity/logs/mtk/d3/manual-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$D3_RUN"
git rev-parse HEAD > "$D3_RUN/host-revision.txt"
git status --short --branch >> "$D3_RUN/host-revision.txt"
sh tools/mtk.sh check > "$D3_RUN/build.log" 2>&1
```

Check the command succeeded and inspect the log:

MAC TERMINAL

```
echo $?
tail -20 "$D3_RUN/build.log"
```

Continue only after exit status `0` and `Project build complete`. If it fails, keep the log and resolve that failure before flashing. Do not change toolchain versions as a workaround.

2. Test the ESP32-S3 alone

Leave all external sensors, jumpers and displays disconnected. Connect the board with a USB data cable. Close other serial monitors.

MAC TERMINAL

```
sh tools/mtk.sh flash > "$D3_RUN/flash.log" 2>&1
```

Check `echo $?` immediately, then `tail -20 "$D3_RUN/flash.log"`. Continue only after success and verified firmware data. Then run:

MAC TERMINAL

```
sh tools/mtk.sh image
sh tools/mtk.sh app-test
sh tools/mtk.sh test 4a
sh tools/mtk.sh test 5-neg
sh tools/mtk.sh test t06
```

Run each separately and stop on a failure. Expected results:

Check	Expected
Image	No FreeRTOS kernel scheduler/start-up code linked
Application	`Application smoke PASS`; synthetic application-path evidence only
`4a`	`14/14 passed`
`5-neg`	`27/27 passed`
`t06`	`7/7 passed`; missing echo is invalid/error, not a distance

The automated parity stages write logs under `parity/logs/mtk/`; copy each result into your new evidence folder before a rerun overwrites it. Save the application smoke log at the path printed by its runner. None of these results proves a real entrance crossing.

Open a recorded console session (macOS):

MAC TERMINAL

```
script -q "$D3_RUN/board-console.log" sh tools/mtk.sh monitor
```

Type these commands **inside the board console**, one per line:

BOARD CONSOLE

```
HELP
TASKS
APP STATUS
```

Check that the board responds without a panic or repeated reboot. ApplicationTask should report core 1, priority 23, `ran_on_core0=0`, and activity on core 1. Save stack-free, heap and drop counts. A low-stack warning or increasing drop count needs investigation. Exit the console with **Ctrl+C** before running another host test.

3. Wire the one sonar on GP4/GP5

Unplug USB. Connect:

HC-SR04+ printed pin	ESP32-S3 printed pin
VCC	3V3
GND	GND
TRIG	GP4
ECHO	GP5, direct for the identified compatible module

Keep radar disconnected for now. Reconnect USB and open a new recorded console:

MAC TERMINAL

```
script -q "$D3_RUN/sonar-console.log" sh tools/mtk.sh monitor
```

Do not run this branch's default `live sonic`, `test 4c-sonic` or `test sonic-rule` against these wires: they use GP1/GP2. The explicit configurations below use GP4/GP5. This checkout does not include the newer `--fixture v2-sonar` option.

4. Check real distance readings

Paste each line into the console separately:

BOARD CONSOLE

```
STOP
CLEAR
BEGINCFG|name=SONAR|proto=gpio|gpio.mode=pulse|gpio.trig=4|gpio.echo=5|gpio.trig_us=10|
gpio.pulse_timeout_us=25000|mode=poll|timing.mode=periodic|timing.interval_ms=500|
timing.timeout_ms=100|ENDCFG
SHOW
RUN
```

Expect `CFG: ok` and recurring `proto=gpio src=SONAR len=3 ...` readings. If configuration is refused, stop and keep the error; do not proceed as if the sensor was added.

1. Hold a flat object roughly 20 cm in front of the sonar for 10 seconds.

2. Move it to roughly 40 cm for 10 seconds, then 80 cm for 10 seconds.

3. Confirm the reported pulse width increases with distance.

The first two hexadecimal bytes are little-endian pulse width in microseconds. Distance in centimetres is approximately `width_us / 58`. For example, `88 13 01` means 5000 µs, about 86 cm. These are sanity checks, not a declared accuracy tolerance.

Type `STOP`. Recurring readings should stop after any already-queued output drains. Type `SHOW` and `TASKS`; the board should still respond. Record the observations.

5. Check timeout, one-shot and recovery

For a definite missing-Echo test, exit the monitor, unplug USB, disconnect all four sonar wires, then power the ESP32 again. Open a fresh recorded monitor. Configure an explicit pulldown on the now-disconnected Echo input:

BOARD CONSOLE

```
STOP
CLEAR
BEGINCFG|name=SONAR|proto=gpio|gpio.mode=pulse|gpio.trig=4|gpio.echo=5|gpio.pull=down|
gpio.trig_us=10|gpio.pulse_timeout_us=25000|mode=poll|timing.mode=one_shot|
timing.timeout_ms=100|ENDCFG
RUN
```

Expect one failed acquisition, normally `[SENSOR] name=SONAR error=-1`, rather than a valid-looking width. Wait a few seconds: one-shot should not keep acquiring by itself. Then type:

BOARD CONSOLE

```
TRIGGER SONAR
```

Expect another single acquisition error. Then:

BOARD CONSOLE

```
STOP
CLEAR
SHOW
TASKS
```

The board must remain responsive and configuration should be cleared. This manual procedure demonstrates timeout/rearming and ordinary STOP/CLEAR recovery. It does **not** establish that STOP/CLEAR landed inside the approximately 25 ms pending-Echo interval; that needs a timed command/capture test.

Exit, unplug USB, reconnect the four sonar wires from step 3, reconnect USB and repeat step 4 to verify recovery with real echoes.

6. Demonstrate event-driven scheduling

With the GP4/GP5 sonar connected, use the console:

```
BOARD CONSOLE
STOP
CLEAR
BEGINCFG|name=TICK|proto=gpio|gpio.mode=digital|gpio.pin=8|gpio.pull=up|mode=poll|
timing.mode=one_shot|ENDCFG
BEGINCFG|name=SONAR|proto=gpio|gpio.mode=pulse|gpio.trig=4|gpio.echo=5|gpio.trig_us=10|
gpio.pulse_timeout_us=25000|mode=poll|timing.mode=event_driven|
timing.trigger_source=TICK|timing.window_ms=100|timing.timeout_ms=100|ENDCFG
RUN
```

GP8 must be physically unconnected; its internal pull-up provides the source reading. After the initial source/sonar activity settles, there should be no continuous sonar stream. Type `TRIGGER TICK` and wait for it to finish before repeating. Each completed source read should arm the sonar. Save the TICK and SONAR output together, then `STOP` and `CLEAR`.

This tests scheduling from another sensor's completed reading. It is not radar-presence triggering. The 100 ms setting is a test configuration, not a new acceptance threshold.

7. Wire and check LD2450 UART data

Exit the console and unplug USB. Leave the sonar connected and add:

LD2450 printed pin	ESP32-S3 printed pin
5V / VCC	5V
GND	GND, common with sonar
TX	RX / GPIO44
RX	TX / GPIO43

Do not connect radar power to the sonar's 3V3 supply. The LD2450 uses a 5 V supply and defaults to 256000-baud UART, one stop bit, no parity; see the manufacturer's manual. This fixture assumes the standard LD2450 UART interface. Use the printed TX/RX labels; USB console remains on USB.

Reconnect USB, open a recorded console and paste:

```
BOARD CONSOLE
STOP
CLEAR
BEGINCFG|name=RADAR|proto=uart|uart.tx=43|uart.rx=44|uart.baud=256000|uart.bits=8|
uart.parity=N|uart.stop=1|uart.line_mode=0|uart.read_len=64|mode=poll|
timing.mode=periodic|timing.interval_ms=20|ENDCFG
RUN
```

Expect recurring `proto=uart src=RADAR ...` byte data. The repository's LD2450 decoder expects 30-byte frames starting `AA FF 03 00` and ending `55 CC`; serial read chunks may split or combine frames, so do not require one output line per frame. Walk in front of the radar and save the data. Raw bytes alone do not prove correct target decoding or presence decisions.

Type `STOP`, then `TASKS`. If there is no data, check power, common ground, crossed TX/RX and whether the radar's baud setting was changed. Do not treat `CFG: ok` alone as evidence that a radar is connected.

For a disconnected-input check, power off, remove the radar connections, then reboot and repeat this configuration. No incoming UART frames should appear, but the console must still respond. Generic UART may return no data without an explicit absent-device error; do not record that as a proven radar fault alarm. Power off before reconnecting and repeat the data check.

8. Run sonar and radar together

Keep both wired as above. In the console:

BOARD CONSOLE

```
STOP
CLEAR
BEGINCFG|name=SONAR|proto=gpio|gpio.mode=pulse|gpio.trig=4|gpio.echo=5|gpio.trig_us=10|
gpio.pulse_timeout_us=25000|mode=poll|timing.mode=periodic|timing.interval_ms=500|
timing.timeout_ms=100|ENDCFG
BEGINCFG|name=RADAR|proto=uart|uart.tx=43|uart.rx=44|uart.baud=256000|uart.bits=8|
uart.parity=N|uart.stop=1|uart.line_mode=0|uart.read_len=64|mode=poll|
timing.mode=periodic|timing.interval_ms=20|ENDCFG
RUN
```

Run for about one minute while moving a flat target in front of the sonar and walking in front of the radar. Confirm both source names keep appearing and the console remains responsive. During and after the run, capture `TASKS`, `SHOW` and `APP STATUS`, then `STOP`.

Record resets, sensor errors, available stack/heap and any exposed drop counters before and after. This is a basic co-running smoke check. It does not prove zero UART loss, fully nonblocking UART service or full-workload timing: the generic UART poll path differs from `WINDOW`'s bounded ready-drain path.

9. Record results and what remains

Copy this table into the evidence folder and fill it from actual output:

Check	Pass / fail / pending	Evidence and observation
Build, flash and normal boot	Pending	
App smoke, 4a, 5-neg, t06	Pending	
ApplicationTask core, stack and heap	Pending	
Real GP4/GP5 distance readings	Pending	
Missing Echo, one-shot and rearming	Pending	
STOP/CLEAR and real-echo recovery	Pending	
Source-triggered scheduling	Pending	
LD2450 UART and disconnected-input recovery	Pending	
Sonar and radar co-running	Pending	

Still separate: exact pending-Echo cancellation; physical boot-counter/power-cycle behaviour; MQTT publication with a broker; agreed requested-versus-measured timing evidence; and queue-overflow/fault qualification. D5 benchmarking is paused under the supplied feedback; do not infer timing acceptance from this short smoke run.

A second sonar is required for physical `PAIR`, acoustic guard/crosstalk and real `WINDOW` entry/exit tests. The present single-sonar setup cannot close those checks. Do not configure the same sonar twice to imitate a pair. The team still needs to confirm whether the doorway counter remains in the final D6 demo.